IMPLEMENTING AN ALGORITHM FOR THE GENERATION OF VISUALLY AND PHYSICALLY REALISTIC CLUMPS OF SNOW IN OPENGL

Mars Semenova
Dalhousie University
April 13, 2026

1. INTRODUCTION

Simulations of natural phenomena in the physical world, such as snowfall, are important in many fields, including computer graphics, gaming, and various engineering disciplines [1]. Particularly in graphics, we see much interest in realistic scenes, and this research would enable the rendering of visually accurate snowfall and its use in the areas of Augmented Reality (AR), Virtual Reality (VR), game development, and digital media. There are various implementations of snow in these areas but they often employ tactics that disregard the properties of snow in the physical world in favour of performance, which is often necessary, especially in particle systems, due to the number of snowflakes that need to be rendered. This research seeks to combine visual realism and physical accuracy in one practical algorithm for snow generation.

This project involved implementing an algorithm for the generation of visually and physically realistic clumps of snow in OpenGL by using T. B. Moeslund et al.'s [2] snow generation model as a reference and running experiments to determine optimal modifications. The original plan was to use the work of C. Zou et al. [3] as the starting point but upon further consideration the idea of smoothing the triangles seemed counterintuitive since snow is naturally angular. Instead, the work this research produces is compared to both models to determine its success. The C. Zou et al. [3] model was not itself implemented since its propositions, which are all centred around the smoothing of the triangles, are not used and it did not make sense to spend a lot of time implementing Bézier curves when there was enough in the paper for comparison. Additionally, the focus on performance was left for future work due to time constraints.

The hypothesis was that the snow generation model described by T. B. Moeslund et al. [2] could be modified to create more visually realistic snow than the results of their work and that of C. Zou et al. [3]. This was proven correct, but while areas of improvement within the model were identified and experimented on, the final result leaves much to be desired.

2. PRIOR WORK

The primary paper guiding this work is B. Moeslund et al. [2], which proposes a snow generation model involving the physical properties of snow and triangles. There is much room for improvement as up close it is clear that it is not all that realistic. This is also a core paper in the field of snow rendering which many have used and expanded on, including C. Zou et al. [3].

C. Zou et al. [3]'s work expands on the model of snow generation proposed by T. B. Moeslund et al. [2] by proposing an algorithm which alters the generation of the triangles making up the clumps of snow and smooths them to achieve a more visually realistic look. Initially, this project sought to improve this model, but since its proposals are counterintuitive regarding snow in the physical world, it seemed more likely to achieve a better snow generation model by using T. B. Moeslund et al. [2]'s work as a base instead of this work. It is still used as a point of comparison to evaluate the success of this project.

The work of P. Goswami [1] is a literature review of snow and ice animation methods in computer graphics. By examining the literature about snow, it is evident that there are gaps in the

generation of visually realistic snow which is also physically accurate and performs well enough to be practical.

3. METHODS

This project involved three steps:

(i) Implementing the T. B. Moeslund et al. [2] snow generation model,
(ii) Determining values which can be modified, and
(iii) Running experiments to determine which values yield the best results in comparison to the T. B. Moeslund et al. [2] model and the C. Zou et al. [3] model.

In the first step, the snow generation model was implemented by generating triangles in layers of concentric spheres (**Figure 1**). The diameter of the clump of snow is dictated by the temperature, fluctuating randomly ±50%. The density of the snowflake is determined by the state of the snow, which is wet if the temperature is above -1℃ or dry if it is below, but this value is not used in the generation of particles. It is not specified what state -1℃ yields so wet snow is assumed.

Figure 1. T. B. Moeslund et al.'s model of snow generation. Source: [3].

T. B. Moeslund et al. [2] do not specify how many layers are generated so 6 is assumed. There are 40 triangles per layer if the snow is wet and 10 if it is dry. The triangles in each layer except the first are generated by randomly choosing a vertex in the previous layer and setting the first vertex of the new triangle to have the same θ and φ, and a ρ of a point that lies within the triangle the random vertex is part of, which is less than the ρ of the random vertex. The second condition for ρ is poorly specified because it may be that the random vertex has the smallest ρ in the triangle so it is ignored. The other two vertices of the new triangle have a θ and φ of the first vertex's $\theta \pm \varepsilon$ and the first vertex's $\varphi \pm \varepsilon$, respectively, where ε is the allowed angular change, set to 80° by T. B. Moeslund et al. [2]. The ρ for each of these two vertices is determined by randomly choosing a ρ between the previous layer and the next layer. It is not specified how the first layer is generated, so it is assumed that the first vertex is generated randomly with a ρ between 0 and the next layer, and the two other vertices are generated the same as in the other layers, except that the θ and φ are of the first vertex.

Additionally, a simple Phong shader was implemented to render the model, but one which would improve realism was left for future work.

Three primary types of parameters which can be modified were determined as part of the second step:

1. Parameters not subject to physics which can be experimentally determined
    - Shininess for wet and dry snow in the Phong lighting model
    - Degree of allowance for changes in θ and φ between vertices in a triangle (ε)
2. Arbitrary constants which should instead depend on physics
    - Opacity of triangles for wet and dry snow (should depend on density)
    - Number of layers (should depend on density)
3. Constants where applying variance would prevent uniformity
    - Opacity of triangles for wet and dry snow

Based on this, the five variables used in experimentation are:

(a) A degree of allowance for changes in θ between vertices in a triangle and a degree of allowance for changes in φ between vertices in a triangle
(b) A function for the opacity of triangles dependent on density and a range of allowed change for random variance
(c) A function for the number of layers dependent on density
(d) A constant for the shininess of wet snow and one for dry snow in the Phong lighting model

For step three, (a) was experimented on first. Starting at 70° for the degree of allowance for θ and 35° for that of φ, three instances of possible values were created by repeatedly decreasing the degree of allowance for θ by 10° and that of φ by 5°. These were then compared to the T. B. Moeslund et al. [2] model, where the variance for both θ and φ was 80°.

Next, a value for (b), to be multiplied by the density, was determined visually by running an instance of wet snow and an instance of dry snow and evaluating whether a difference in the two added realism compared to the wet snow and dry snow generated by T. B. Moeslund et al.'s [2] model. After the optimal opacity coefficient was determined, eight more instances, four for wet and four for dry snow, were created to compare no variance in opacity with variances of ±10%, ±25%, and ±50%.

For (c), since the number of layers has to be an integer and the difference between the number of layers for wet snow and the number of layers for dry snow should be at most 2 to avoid unnatural differences, it was decided that the function would be a binary threshold based on snow state. A pair of values was determined visually by comparing 4 layers for wet snow and 6 layers for dry snow, 5 layers for wet snow and 6 layers for dry snow, and 5 layers for wet snow and 7 layers for dry snow. The pair with the most natural difference between the wet snow and dry snow was chosen.

Lastly, (d) was determined by trying various values for both wet and dry snow and evaluating which looked best in practice and in comparison to each other.

It is worth noting that at each step, if a variable was not determined yet, it was set to its equivalent value in the T. B. Moeslund et al. [2] model. Once a variable was determined, it was updated in the application constants and consequent instances of snow generated using the experimental algorithm used these values.

4. RESULTS

For each of the variables outlined in the methods section, instances to compare were generated and the best one was chosen visually.

Figure 2 shows the comparisons for (a). Reducing the degree of allowance and setting different yet proportional allowances for θ and φ yielded an improvement. It seems that for wet snow 60° of allowance for θ and 30° of allowance for φ was best, and for dry snow 70° for θ and 35° for φ was best. These values generated triangles that were less sharp than those in T. B. Moeslund et al.'s [2] model, which was the goal, and they were neither too crowded nor too spread apart.

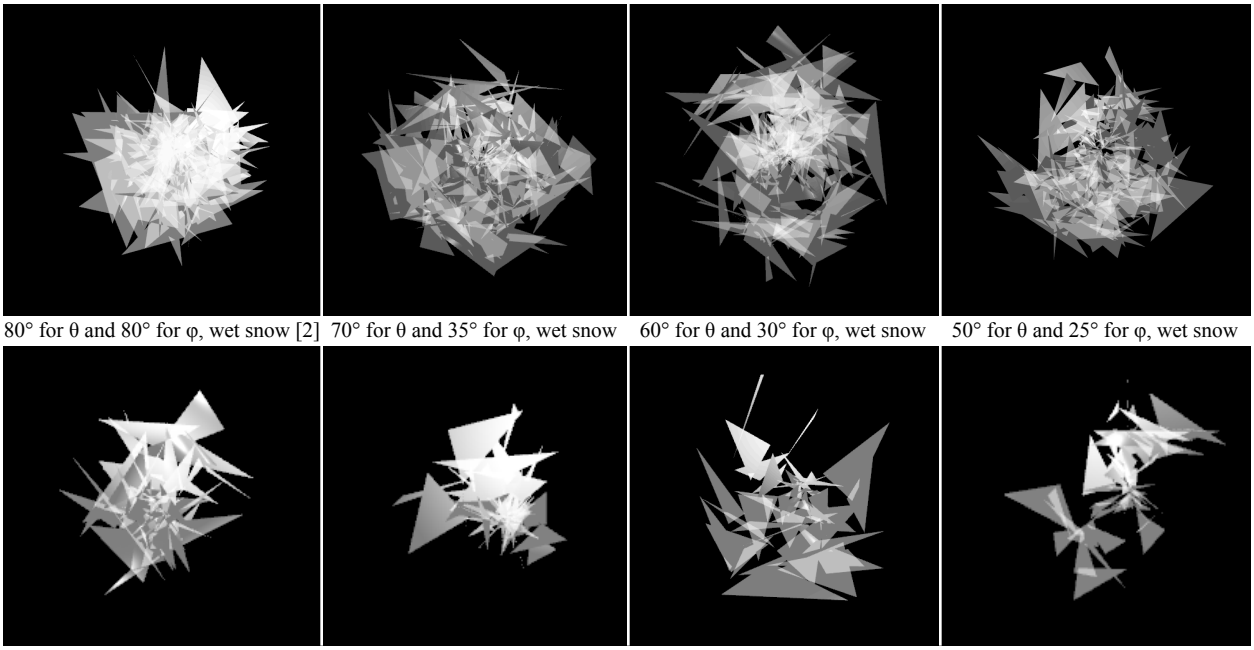
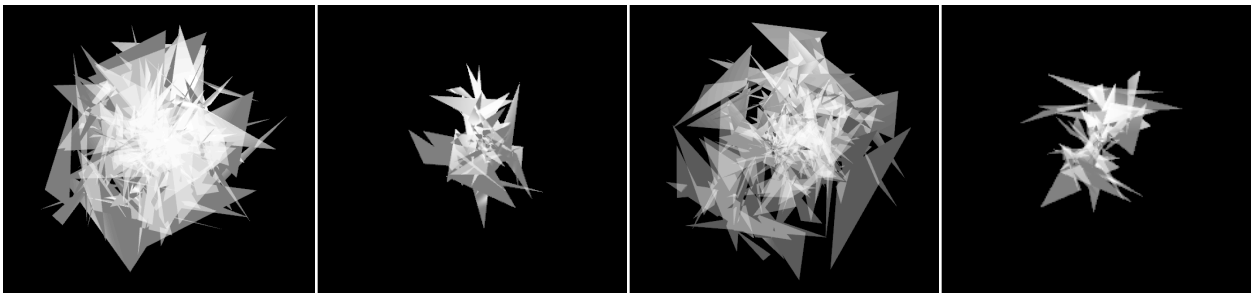


Figure 2. Comparisons for the degrees of allowance for changes in θ and φ between vertices in a triangle.

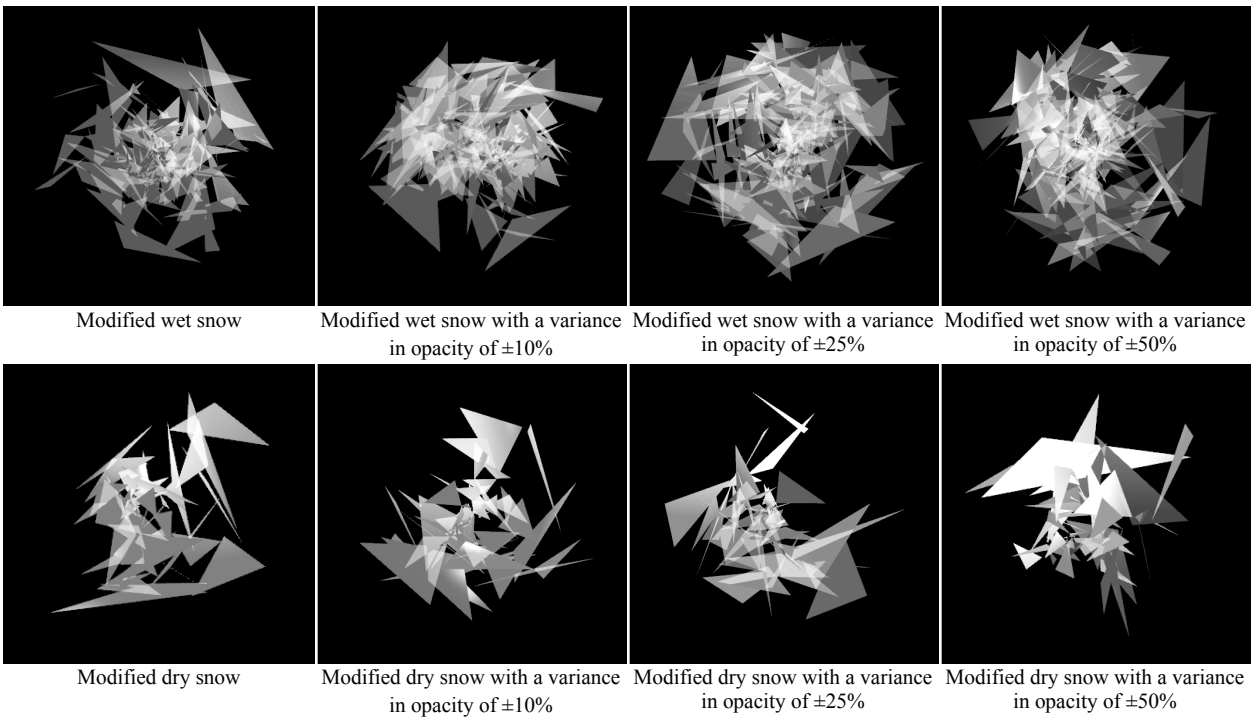
Figure 3 shows the comparisons for the opacity coefficient in (b), which was set to 0.02. The opacity function did not make much of a difference, but perhaps a more sophisticated formula than simple multiplication, potentially even one driven by physics, could be determined.



T. B. Moeslund et al. [2] wet snow | T. B. Moeslund et al. [2] dry snow | Modified wet snow | Modified dry snow

Figure 3. Comparisons of the effect of an opacity function based on density.

Figure 4 shows the comparisons for the variance in the opacity generated by the function in (b). Adding a degree of randomness to the opacity of triangles improved the result, with a variance of ±25% for both wet and dry snow being optimal. A higher variance yielded too much contrast to be realistic.

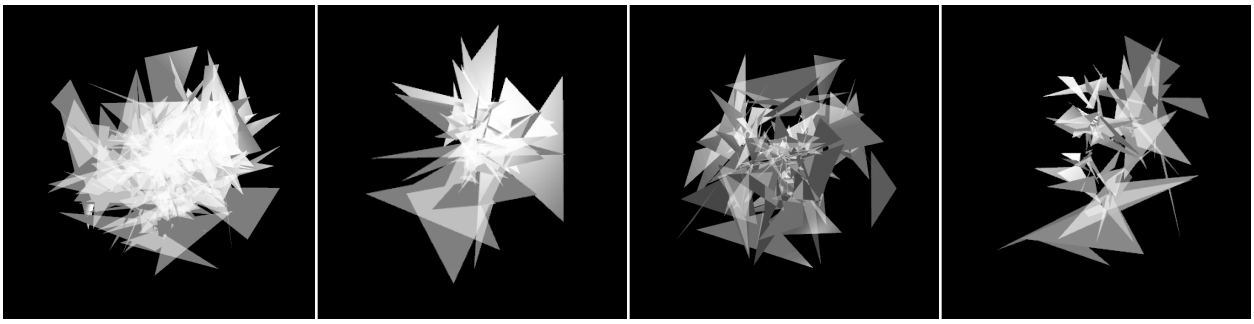


Modified wet snow | Modified wet snow with a variance in opacity of ±10% | Modified wet snow with a variance in opacity of ±25% | Modified wet snow with a variance in opacity of ±50%

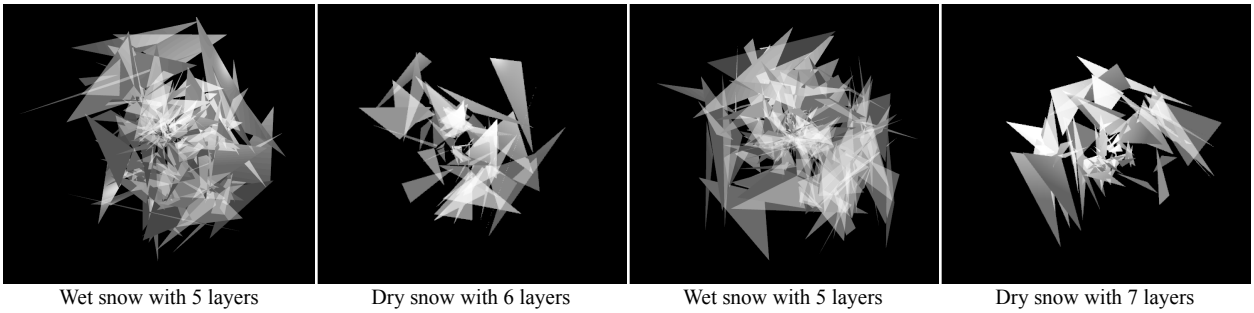
Modified dry snow | Modified dry snow with a variance in opacity of ±10% | Modified dry snow with a variance in opacity of ±25% | Modified dry snow with a variance in opacity of ±50%

Figure 4. Comparisons for the variance in opacity.

Figure 5 shows the comparisons for (c), with 5 layers for wet snow and 6 layers for dry snow determined to be optimal. A difference of layers between the two states of snow resulted in an improvement but any less or any more of each caused either crowding or sparseness.



T. B. Moeslund et al. [2] wet snow | T. B. Moeslund et al. [2] dry snow | Wet snow with 4 layers | Dry snow with 6 layers



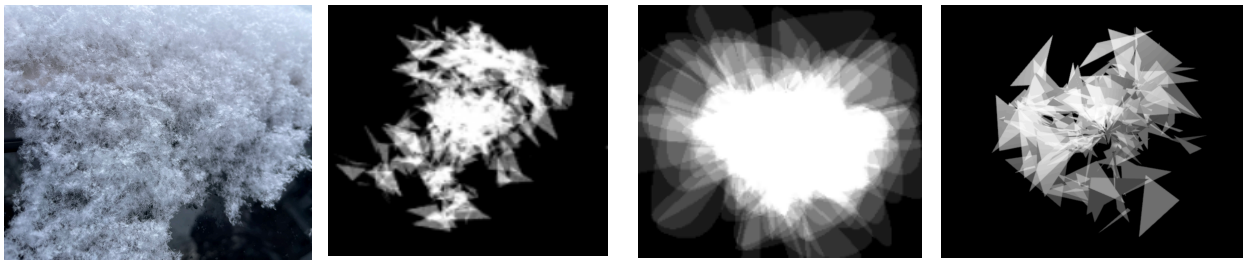
Wet snow with 5 layers | Dry snow with 6 layers | Wet snow with 5 layers | Dry snow with 7 layers

Figure 5. Comparisons for the number of layers for wet snow and the number of layers for dry snow.

In the process of generating comparisons for (d), it was found that the shininess coefficient did not affect the model at all. This is likely because of the low opacity and the simplicity of the shader. Further work on the shader would likely make better use of the shininess coefficient.

5. CONCLUSIONS

The results of this project are determined visually by comparing a generated snow particle with the experimentally determined values to those in T. B. Moeslund et al. [2] and C. Zou et al. [3] and evaluating their up-close appearance regarding realism. This comparison is illustrated in **Figure 6**.



Real snow. Source: [4]. | T. B. Moeslund et al. [2] snow | C. Zou et al. [3] snow | Modified snow

Figure 6. The overall comparison of real snow and the T. B. Moeslund et al. [2], C. Zou et al. [3], and the modified models of snow generation.

This work is definitely a step in the right direction of improving the T. B. Moeslund et al. [2] snow generation model but more rigorous experimentation needs to be done.

Some limitations of this research include the difficult nature of evaluating results, as well as the great variation between generated particles when running experiments in the current configuration.

6. FUTURE WORK

There were several important tasks left incomplete which are crucial to adding realism, such as the implementation of a realistic shader and the design of an algorithm to prevent generated triangles from intersecting unnaturally. Additionally, the realism could be further evaluated by running more rigorous experiments and conducting user studies. There are also some parameters, such as the number of polygons per layer and variations in layer height, which could be experimented on that this work does not address. Experimentation could also be improved by ensuring instances in the same experiment use the same generated values for more similar comparisons.

A vital aspect of this research is its practicality, which is why it is important to optimize it. One of the first steps in doing so would be to parallelize the code and to implement the algorithm on the GPU, which was unfortunately out of scope for this project.

7. IMPLEMENTATION

The code is also available on GitHub. The API is implemented in `SnowGenerator.hpp`, and `SnowRenderer.hpp` shows an example of using the API. Relevant constants are defined in `SnowConstants.hpp`. The API works by instantiating a `SnowGenerator` object with a given temperature which then enables a user to call an assortment of methods to generate snow by specifying the number of particles to generate and the extent of the volume within which snow is generated. The algorithm used to generate the snow is chosen by calling a method for the desired algorithm.

The experimentation is implemented in `SnowGeneratorExperimentation.hpp` and can be run by setting the `whichExperiment` parameter in the constructor to constants associated with experiments (see `README.md` for these constants). This has been implemented in the runner (`main.cpp`) as part of the hard-coded input parameters, and the specified experiment will be run if `test` is set to `true`.

Since running the application locally is extremely cumbersome I have included videos under `/docs/videos/`.

8. BIBLIOGRAPHY

[1] P. Goswami. “Snow and Ice Animation Methods in Computer Graphics”. In: *Computer Graphics Forum* 43 (2 2024). DOI: 10.1111/cgf.15059.

[2] T. B. Moeslund et al. “Modeling Falling and Accumulating Snow”. In: *Second International Conference on Vision, Video and Graphics*, *VVG 2005*. Second International Conference on Vision, Video and Graphics. European Association for Computer Graphics, 2005, pp. 61–68. ISBN: 3905673576. URL: https://www.researchgate.net/publication/267400785_Modeling_Falling_and_Accumulating_Snow.

[3] C. Zou et al. “Algorithm for generating snow based on GPU”. In: *ICIMCS ’10: Proceedings of the Second International Conference on Internet Multimedia Computing and Service*. Association for Computing Machinery, 2010, pp. 199–202. ISBN: 9781450304603. DOI: 10.1145/1937728.1937775.

[4] R. Matoush, “,” *FOX21 News*, Jan. 2024. Accessed: April 13, 2026. Available: https://www.fox21news.com/weather/the-science-of-snow-what-made-fridays-snow-so-fluffy/.